小さなGPTを訓練する

32万字を流し込んで、火を入れる

rkskmt

前回の振り返り

- GPT の全身が完成した：埋め込み → ブロック×2 → 語彙スコア。**約82万パラメータ**
- ただし中身はまだ乱数 — 今日、『吾輩は猫である』**32万字**を流し込んで学習させる

> **ⓘ ノート**
>
> 今日から前回組んだクラス一式を **`gpt.py`** として使う。 **gpt.py** をダウンロードして、自分のスクリプトと同じフォルダに置く。

訓練データの形 — 1枚で64問

- 文章を **64文字**（**`block_size`**）ずつ切り出し、「1文字ずらし」を正解にする

Code

```
import torch

text = open("data/neko.txt", encoding="utf-8").read()
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}

data = torch.tensor([stoi[c] for c in text])
n = int(len(data) * 0.9)
train_data, val_data = data[:n], data[n:]    # 採点用の1割は隠しておく

x = data[:8]                                  # 短い例で仕組みを見る
y = data[1:9]
print("入力:", "".join(chars[i] for i in x))
print("正解:", "".join(chars[i] for i in y))
```

Result

```
入力: 一

  吾輩は猫
正解:

  吾輩は猫で
```

- 入力の各位置に「その次の文字」が正解として並ぶ
- 因果マスクのおかげで、**64文字の1枚が「次を当てる問題」64問ぶん**になる — 前回のマスクはこのための仕込みだった

ミニバッチを切り出す

Code

```
def get_batch(data, block_size=64, batch_size=32):
    ix = torch.randint(len(data) - block_size - 1, (batch_size,))
    x = torch.stack([data[i:i + block_size] for i in ix])
    y = torch.stack([data[i + 1:i + block_size + 1] for i in ix])
    return x, y

xb, yb = get_batch(train_data)
print(xb.shape, yb.shape)     # (32文, 64文字) が2組
```

Result

```
torch.Size([32, 64]) torch.Size([32, 64])
```

- ランダムな一部だけで1歩、の**ミニバッチ**をここでも使う。1バッチ＝32×64＝**2,048問**
- **`torch.stack`** は同じ形のテンソル（tensor）を**重ねて1段上の次元を作る**：64文字の棒32本 → (32, 64) の板

火入れ — 300回だけ回してみる

Code

```
import torch.nn.functional as F
from gpt import GPT

torch.set_num_threads(1)                        # CPUではこの方が速いことが多い
torch.manual_seed(42)
model = GPT(vocab_size=len(chars), n_embd=96, n_head=4, n_layer=2, block_size=64)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)

for it in range(1, 301):
    xb, yb = get_batch(train_data)
    opt.zero_grad()
    logits = model(xb)                                      # (32, 64, 3045)
    loss = F.cross_entropy(logits.view(-1, len(chars)),    # 2,048問まとめて採点
                           yb.view(-1))
    loss.backward()
    opt.step()
    if it in (1, 100, 200, 300):
        print(f"更新 {it:3d}回 | Loss: {loss.item():.4f}")
```

Result

```
更新   1回 | Loss: 8.2259
更新 100回 | Loss: 4.9774
更新 200回 | Loss: 4.2422
更新 300回 | Loss: 4.1335
```

- **3拍子**の学習ループは**一字一句このまま** — モデルが GPT になっただけ
- **`view(-1, V)`** は並べ直しだけ：(32, 64, 3045) の答案の束を「2,048行の答案リスト」にして交差エントロピーへ
- 初期損失は約8.2（$= \ln 3045$、何も知らない状態の罰点）。そこから一気に転がり落ちる

生成してみる — 300回時点の実力

Code

```
@torch.no_grad()
def generate(model, prompt, n):
    idx = torch.tensor([[stoi[c] for c in prompt]])
    for _ in range(n):
        logits = model(idx[:, -64:])[:, -1, :]   # 見えるのは直近64文字まで
        probs = F.softmax(logits, dim=-1)
        idx = torch.cat([idx, torch.multinomial(probs, 1)], dim=1)
    return "".join(chars[i] for i in idx[0].tolist())

torch.manual_seed(0)
print(generate(model, "吾輩は", 80))
```

Result

```
吾輩はある。何味へ本からるは食うだんがいて元も主人が厭を…死の湯のごと、超吾輩分化なん一来出であると云うと
行る前将に田のして武濶士だ、飴る悟衛時はよこは驚りま早ってで
```

- 自己回帰の生成ループは**バイグラムの回と同じ考え方**：出た文字を入力の尻に足して繰り返す
- まだ怪しい。だがもう「日本語の音」はしはじめている — ここから先は回数の勝負

言葉を覚えていく定点観測

同じスクリプト（**train_neko.py**）を回し続け、節目ごとに生成させた記録：

ⓘ 0回（乱数のまま）

寺吹瞳麼Q次軒融亡項多忙鴻武箔未肯美汚へ煙促喪襟欣銷京紫胎篤銅狎撮腹杭棄羊骨絞煉果物孤葵層幣穴姿他娘誦磯持……

ⓘ 50回 — ひらがなと句読点を発見

だなり大ではて融んて多こょ武箔未そに汚。をるしずせ銷っの大が御や。ててて時す。な果物のとう幣てる今…い。持ののてあ師合面たでけっん……

ⓘ 500回 — 言葉のかけらが出る

「なりゃでは、融んだ多忙いわんです。汚へを促しずせている大き御やり腹の男時はなさ。物を受けると云っている。第存に待し合面白いけに区物を食え」……

ⓘ 2,000回 — 文の形になる

果物を受けるところだと椽側のごとく眼を面白いかに区別らしておく。群鶴館なのみなんだ。それから感心みたら君の下へはもっとよい切って行きっと正おると間違える。吾輩は……

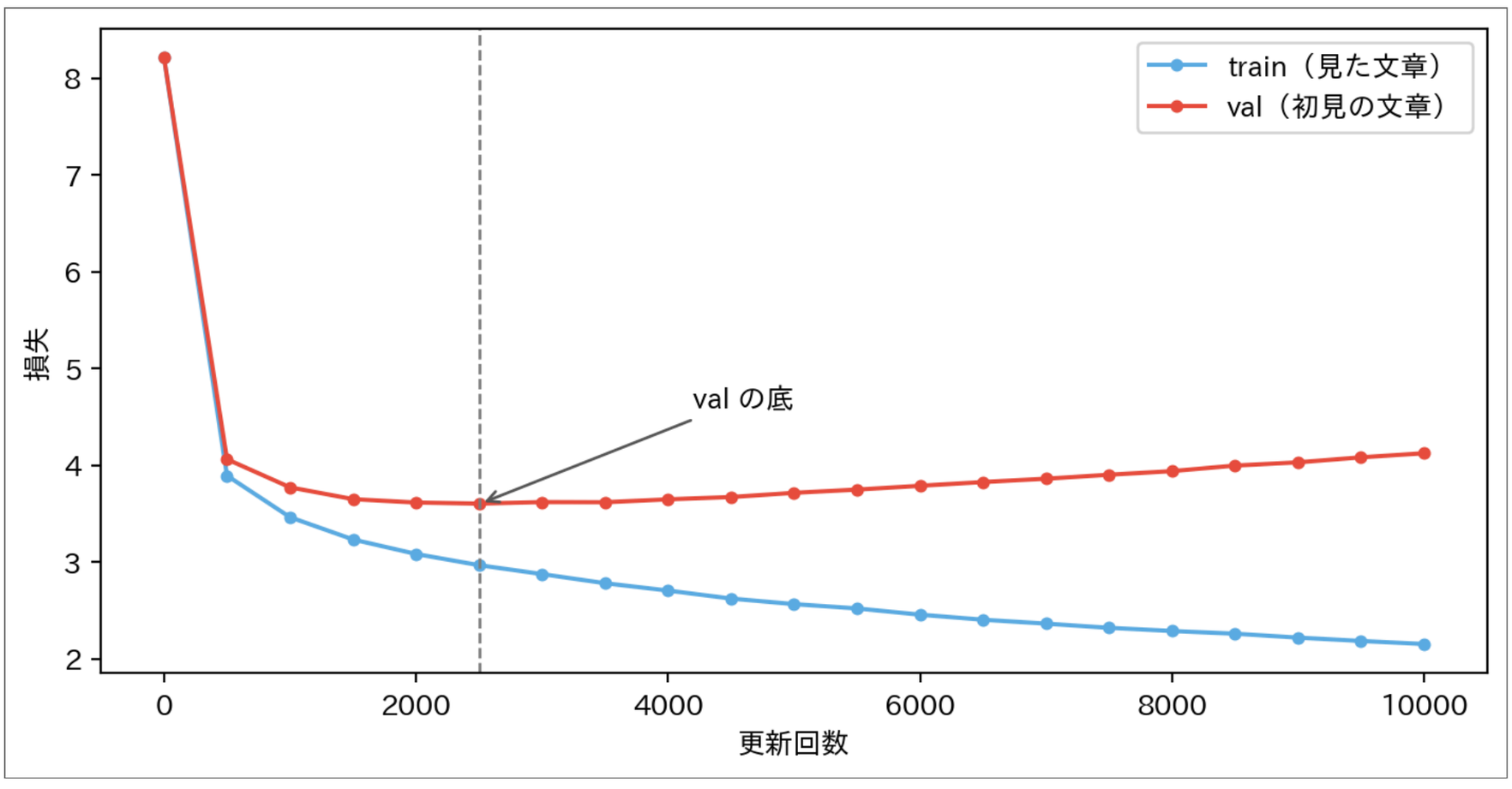
ⓘ 10,000回（約50分）

「なるほどは、融突然多忙ですよ。それじゃ、私しゃボイがメンチボーの御師匠さんが、果物を一層の通り今日に至るまで遠征を入れた時改急に一生懸命になって上げなければ……

- 文字 → かな → 単語 → 文 — **人間の子どもと同じ順番**で言葉を覚えていく

クイズ：10,000回回すと何が起きる？

> **クイズ**
>
> 損失のグラフはどうなっていると思う？ 訓練データ（見た文章）と、隠しておいた1割（初見の文章）の**2本の線**を想像してから、めくる。



- **val は2,500回あたりで底（約3.6）を打ち、その後は悪化** — train だけが下がり続ける
- 32万字を10,000回で**約64周**も読んだ（$10000 \times 2048 \div 321139$）— 同じ本の読みすぎ

過学習、最終決戦

- 地名MLPの回とまったく同じ形（**過学習**）が、GPT でも出た — **モデルが変わっても病気は同じ**
- 訓練文の細かい癖に最適化が進み、**初見の文章への実力はむしろ落ちていく**
- 対策も同じ発想：**val が底を打ったところで止めて、その時点の重みを使う** — **早期終了**（**early stopping**）と呼ぶ

> **ノート**
>
> 本物の LLM はこの逆で「同じ文章を何周もしない」規模のデータを使う — データの量で過学習をねじ伏せている（この話は「ChatGPTまでの距離」の回で）。

訓練済みの重みを読み込む

- 訓練は途中の重みを保存できる — この保存ファイルが**チェックポイント**（checkpoint）。val の底で保存した配布用を用意した
- **ckpt_best.pt**（3.4MB）をダウンロードして **data/runs/neko/** に置く

Code

```python
import torch
from gpt import GPT

blob = torch.load("data/runs/neko/ckpt_best.pt", weights_only=True)
model = GPT(vocab_size=len(blob["vocab"]), **blob["cfg"])
model.load_state_dict(blob["state_dict"])    # 82万個の数を一括で流し込む
model.eval()

chars = blob["vocab"]
stoi = {c: i for i, c in enumerate(chars)}
torch.manual_seed(1)
print(generate(model, "吾輩は", 100))
```

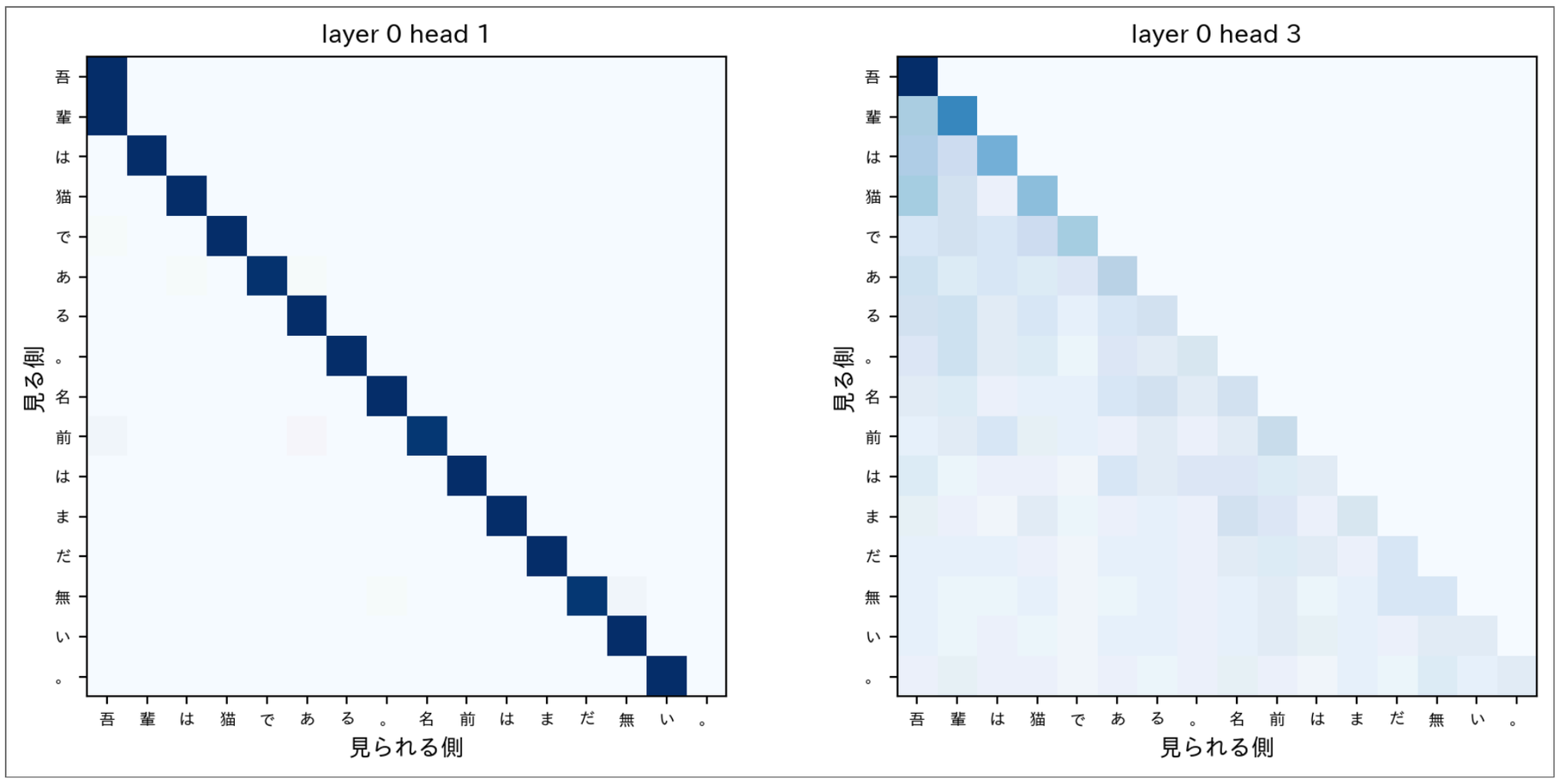
Result

```
吾輩は最後屁ぽために獣に不平を洩れたまって……」「うん、どうだなに蝙蝠傘に話し給え。あんまりあの」と心配して、食べてこの質問をは弁当人の不平を少しか忘れている。
　芸者はやはりかず、落ちておこちへ降がって聞
```

- 同じ **train_neko.py** を3,500回（このPCで約20分）回しただけの重み。**魔法の材料はゼロ**

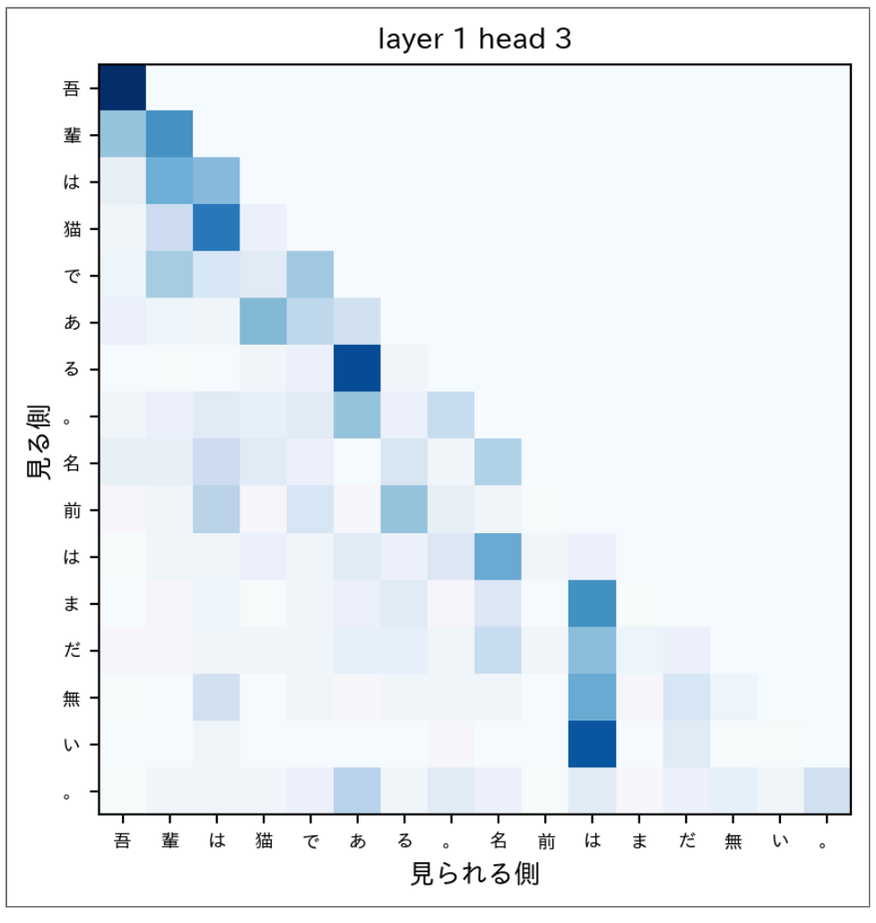
育った視線を覗く

- 「束ねて、考える」の回の宿題を回収する — あのとき**どのヘッドがどんな「係」（視線の分業）になるかは学習まかせ**と約束した
- 訓練済みモデルに冒頭の一文を読ませ、層×ヘッドの**視線の表**（Q・K・V の回で作った「誰が誰を見るか」の表）を取り出して眺める



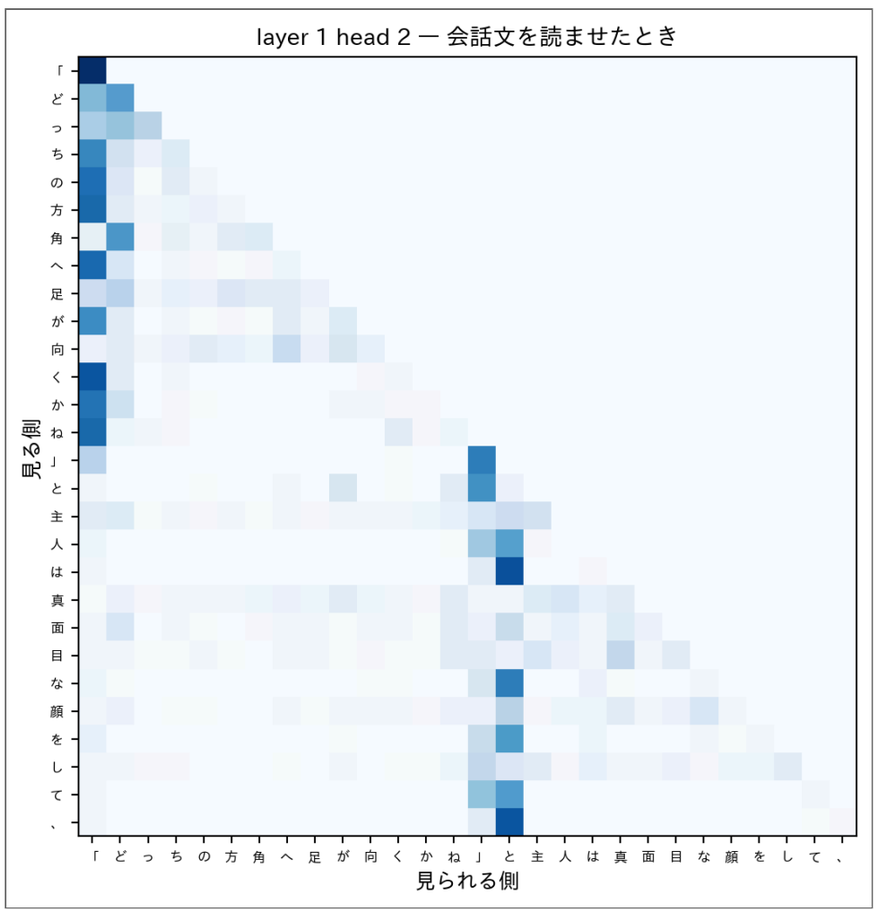
- **左（head 1）は副対角線1本**：どの文字も**直前の1文字だけ**を重み約99%で見る係 — バイグラムの再発明
- **右（head 3）は遠くまで広く薄く**見る係 — 同じ式・同じデータから、正反対の分業が勝手に育った

「名前はまだ」の視線



- 「…名前はまだ→（？）」を予測する瞬間（「だ」の行）、この係の視線は**「は」に41%、「名」に22%**
- 注目という発想の回の手計算（「まだ」から「名前」へ視線が飛ぶ）が、**実物の中で本当に起きている**

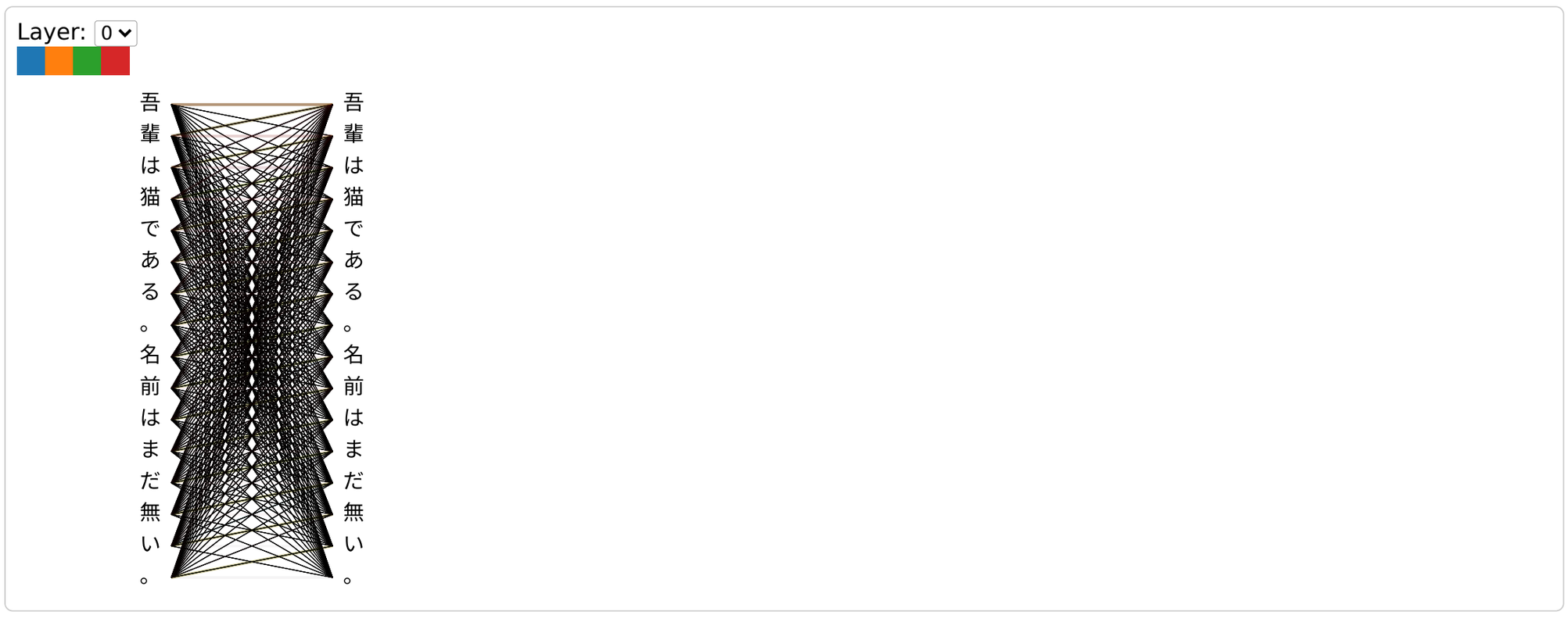
14文字前のカッコを見る係



- 「「」と「と」の列に**縦の縞** — カッコ・引用の構造を追いかける係
- 「」」の位置からは、**14文字前の「「」に28%**の視線。「、」からは12文字前の「と」に85%
- **窓では届かない距離を、位置ではなく内容で見ている** — 固定の窓を捨てて手に入れたものが、ここに写っている

視線と対話する — BertViz

- 静止画は1枚ずつしか見られない。**BertViz** というライブラリなら、トークンをなぞって全ヘッドの視線を対話的に眺められる（下の埋め込みで今すぐ試せる）



▸ 自分の手元で動かすコード（pip install bertviz が必要）

- 好きな文（語彙にある文字だけ）に差し替えて、**お気に入りのヘッドを1つ見つけて「係の名前」を付ける** — 隣の人と見つけた係を交換
- 線の濃さ＝視線の重み。左の列でレイヤを切り替えられる

> **ⓘ ノート**
>
> 表示には d3.js などを CDN から読むためネット接続が要る。**`pip install bertviz`** は初回のみ。

全部品を一望する — 自作GPTエクスプローラ

視線だけでなく、**全部品の中身**を歩ける。埋め込みから次の1文字の予想まで、ぜんぶ訓練済みの実物の数値。



0. いま開いているのは「吾」だけ読んだ位置の予想 — 1位は「輩」**71.4%**（ことばの地図の回の「吾→輩」がここでも生きている）
1. 文字を順にクリックして、予想が文脈で変わっていく様子を歩く
2. 途中の部品（LayerNorm・FFN・残差）も開いて、ヒートマップの模様の移り変わりを眺める
3. 隣の人に、好きな部品を1つ選んで「この箱は◯◯をしている」と30秒で説明

エクスプローラを別タブで開く

Tweak 火加減を変える

- **300回訓練のコード**を書き換えて、生成まで続けて実行

1	`range(1, 301)` → `range(1, 1001)`	3倍待つと生成文はどれだけ変わる？（時間も測ってみる）
2	`batch_size=32` → `8`	1回が速くなる代わりに、Loss の下がり方はどうなる？
3	`generate(model, "猫は", 80)`	書き出しを変える。語彙にある文字なら何でも使える

ここで観察

- 訓練を伸ばすほど良くなるのは（このサイズとデータでは）**val の底まで**。「長く回す＝偉い」ではないことをグラフが示している

まとめ

- 訓練データ＝「64文字の窓」と「1文字ずらしの正解」。マスクのおかげで1枚が64問になる
- 学習ループは3拍子のまま。損失 8.2（無知）→ 3.6（val の底）
- 生成の定点観測：文字 → かな → 単語 → 文 の順に覚えていく
- **過学習は GPT でも起きる**。val を見張り、底の重みを使う（早期終了）
- 育った視線は**係に分かれていた**：直前の1文字係・広く見る係・カッコ係 — マルチヘッドの分業は本物だった

現在の設計図　訓練窓 → GPT → 損失 → **backprop・更新・checkpoint**

ⓘ 次回予告

訓練済みモデルを手に入れた。次回は**生成のツマミ**（temperature・top-k）を握って、「無難な文」と「壊れた文」のあいだを自由に行き来する。

練習問題

以下の問題を**Pythonコード**で解け。

問題1：何周読んだか

バッチ32本×64文字で10,000回更新したとき、モデルが処理した総文字数を計算し、『吾輩は猫である』（321,139字）の**何周ぶん**にあたるか求めよ。500回・3,500回ならそれぞれ何周か。

問題2：チェックポイントの中身

`ckpt_best.pt` を読み込み、以下を表示せよ。

1. 保存されている語彙のサイズ（**`len(blob["vocab"])`**）
2. モデル設定（**`blob["cfg"]`**）
3. **`state_dict`** の中で**一番大きい**パラメータの名前と形（**`blob["state_dict"].items()`** を **`numel()`** で比べる）

解答例

▶ 解答例を見る

Result

```
   500回 →    3.2 周
  3500回 →   22.3 周
 10000回 →   63.8 周
語彙: 3045 | 設定: {'n_embd': 96, 'n_head': 4, 'n_layer': 2, 'block_size': 64}
最大のパラメータ: tok_emb.weight (3045, 96)
```

- 問題1：3,500回でもすでに**22周**。データより回数が多い訓練は、いつか暗記に向かう
- 問題2：最大は文字の埋め込み表（3045×96）— 実は出口の語彙スコアの行列も同じ大きさで同率1位。大語彙のコストがここでも主役